Loops in Java

Beginner-friendly notes | DSA in Java Series

1. What are Loops?

- A loop **repeats a block of code** again and again while a condition is true.
- It saves you from writing the same code many times.
- Each time the block runs once, it is called an **iteration**.
- The loop stops when the condition becomes **false**.

> Without a loop, printing numbers 1 to 100 needs 100 print statements. With a loop, it needs only 3 lines.

Java has three basic loops:

Loop	Best used when
for	You know **how many times** to repeat
while	You do **not know** how many times, but you know the condition
do-while	The code must run **at least once**

2. The for Loop

Syntax

```
for (initialization; condition; update) {
    // code to repeat
}
```

Part	Meaning	Example
Initialization	Sets the starting value. Runs **only once**	int i = 1
Condition	Checked before every iteration. Loop runs while it is true	i <= 5
Update	Changes the variable after every iteration	i++

How it runs, step by step

- **Step 1:** Initialization runs once.
- **Step 2:** The condition is checked. If false, the loop ends.
- **Step 3:** If true, the body runs.
- **Step 4:** The update runs, then go back to Step 2.

Example: print numbers 1 to 5

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Output: 1 2 3 4 5 (each on a new line)

Dry run (trace table)

Iteration	Value of i	i <= 5 ?	Action
1	1	true	prints 1, then i becomes 2
2	2	true	prints 2, then i becomes 3
3	3	true	prints 3, then i becomes 4
4	4	true	prints 4, then i becomes 5
5	5	true	prints 5, then i becomes 6
-	6	false	loop ends

Common update styles

```
for (int i = 1; i <= 10; i++)   { }   // count up by 1
for (int i = 10; i >= 1; i--)   { }   // count down by 1
for (int i = 0; i <= 20; i += 2) { }  // jump by 2 (0, 2, 4 ...)
```

The variable declared inside the for loop (like **i**) exists **only inside that loop**. You cannot use it after the loop ends.

3. The while Loop

Syntax

```
initialization;
while (condition) {
    // code to repeat
    update;
}
```

- The condition is checked **first**. If it is false at the start, the body never runs.
- You must write the initialization **before** the loop and the update **inside** it.
- If you forget the update, the loop never ends.

Example: print numbers 1 to 5

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Output: 1 2 3 4 5 (each on a new line)

Example: sum of digits of a number

```
int num = 1234, sum = 0;

while (num > 0) {
    sum = sum + num % 10;   // take the last digit
    num = num / 10;         // remove the last digit
}
System.out.println(sum);
```

Output: 10

4. The do-while Loop

Syntax

```
initialization;
do {
    // code to repeat
    update;
} while (condition);
```

- The body runs **first**, and the condition is checked **after**.
- So the body always runs **at least once**, even if the condition is false.
- Do not forget the **semicolon ;** after the while condition.

Example: the condition is false from the start

```
int i = 10;
do {
    System.out.println("i = " + i);
    i++;
} while (i < 5);
```

Output: i = 10 (it ran once even though 10 < 5 is false)

Typical use: menus and input validation, where you must show something at least once.

5. for vs while vs do-while

Point	for	while	do-while
Type	Entry controlled	Entry controlled	Exit controlled
Condition checked	Before the body	Before the body	After the body
Minimum runs	0	0	**1**
Best for	Known number of repeats	Unknown number of repeats	Must run at least once
Semicolon at end	No	No	Yes, after while( )

6. break and continue

These two statements change the normal flow of a loop.

Statement	What it does
break	**Stops the loop completely** and jumps to the code after it
continue	**Skips the current iteration** and goes to the next one

break example

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.print(i + " ");
}
```

Output: 1 2 3 4

continue example

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.print(i + " ");
}
```

Output: 1 2 4 5 (3 is skipped)

Inside nested loops, **break** and **continue** affect only the **innermost loop** they are written in.

7. Infinite Loops

- An infinite loop is a loop whose condition **never becomes false**.
- It happens by mistake (forgetting the update) or on purpose (using **break** to exit).
- If your program hangs, press **Ctrl + C** in the terminal to stop it.

```
// Mistake: i never changes, so the loop never ends
int i = 1;
while (i <= 5) {
    System.out.println(i);
}

// On purpose: exits using break
while (true) {
    // do something
    if (someCondition) {
        break;
    }
}

// Another infinite form
for (;;) { }
```

8. Nested Loops

- A nested loop is a **loop inside another loop**.
- For every single iteration of the **outer** loop, the **inner** loop runs completely.
- If the outer loop runs 3 times and the inner loop runs 4 times, the inner body runs 3 x 4 = **12** times.
- Nested loops are mainly used to print **patterns** and work with rows and columns.

Example: right triangle pattern

```
for (int i = 1; i <= 4; i++) {          // outer loop: rows
    for (int j = 1; j <= i; j++) {      // inner loop: stars in the row
        System.out.print("* ");
    }
    System.out.println();               // move to next row
}
```

Output:

```
*
* *
* * *
* * * *
```

Example: number pattern

```
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j + " ");
    }
    System.out.println();
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
```

Tip for patterns: the **outer loop** controls the **rows**, the **inner loop** controls the **columns**, and **println()** after the inner loop moves to the next row.

9. Important Programs using Loops

Sum of first N numbers

```
int n = 5, sum = 0;
for (int i = 1; i <= n; i++) {
    sum += i;
}
System.out.println(sum);
```

Output: 15

Factorial of a number

```
int n = 5;
long fact = 1;
for (int i = 1; i <= n; i++) {
    fact *= i;
}
System.out.println(fact);
```

Output: 120 (5! = 5 x 4 x 3 x 2 x 1)

Multiplication table

```
int n = 5;
for (int i = 1; i <= 10; i++) {
    System.out.println(n + " x " + i + " = " + (n * i));
}
```

Output: 5 x 1 = 5, 5 x 2 = 10, ... , 5 x 10 = 50

Count digits of a number

```
int num = 12345, count = 0;
while (num != 0) {
    num = num / 10;
    count++;
}
System.out.println(count);
```

Output: 5

Reverse a number

```
int num = 1234, rev = 0;
while (num > 0) {
    int digit = num % 10;
    rev = rev * 10 + digit;
    num = num / 10;
}
System.out.println(rev);
```

Output: 4321

Palindrome number

```
int num = 121, original = num, rev = 0;
while (num > 0) {
    rev = rev * 10 + num % 10;
    num = num / 10;
}
if (original == rev) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not a palindrome");
}
```

Output: Palindrome

Fibonacci series (first 7 terms)

```
int a = 0, b = 1;
for (int i = 1; i <= 7; i++) {
    System.out.print(a + " ");
    int next = a + b;
    a = b;
    b = next;
}
```

Output: 0 1 1 2 3 5 8

Check prime number

```
int n = 17;
boolean isPrime = true;

if (n <= 1) {
    isPrime = false;
}
for (int i = 2; i <= n / 2; i++) {
    if (n % i == 0) {
        isPrime = false;
        break;
    }
}
System.out.println(isPrime ? "Prime" : "Not prime");
```

Output: Prime

Ideas used here: **% 10** gives the last digit, **/ 10** removes the last digit, and a **boolean flag** with **break** stops the loop early once the answer is known.

10. Common Mistakes to Avoid

- Forgetting the **update** (i++), which creates an infinite loop.
- Putting a **semicolon** right after for(...) or while(...). The loop then does nothing and the block runs only once afterward.
- Off-by-one errors: **i < 5** runs 4 times from 1, but **i <= 5** runs 5 times.
- Using **=** instead of **==** inside a condition.
- Using the loop variable outside the for loop where it was declared.
- Wrong direction: **i--** with a condition that is already false.
- Using **int** for big factorials. int overflows after 12!, so use **long** (it is still limited to about 20!).

Wrong vs right

```
// Wrong: semicolon ends the loop immediately
for (int i = 1; i <= 5; i++);
{
    System.out.println("Hi");   // prints only once
}

// Right
for (int i = 1; i <= 5; i++) {
    System.out.println("Hi");   // prints 5 times
}
```

11. Practice Programs

Try writing these yourself, from easy to slightly harder:

- Print numbers from 1 to N
- Print numbers from N down to 1
- Print all even numbers between 1 and 50
- Find the sum of the first N natural numbers
- Find the sum of all even and all odd numbers up to N
- Print the multiplication table of any number
- Find the factorial of a number
- Count the digits of a number
- Find the sum of the digits of a number
- Reverse a number and check if it is a palindrome
- Print the Fibonacci series up to N terms
- Check whether a number is prime
- Print all prime numbers from 1 to 100
- Calculate x raised to the power y (without Math.pow)
- Print a star triangle and a number triangle

12. Quick Summary

- A loop repeats code while a condition is true.
- **for**: use when the number of repeats is known.
- **while**: use when it depends on a condition. The condition is checked first.
- **do-while**: runs **at least once**. The condition is checked after.
- **break** stops the loop. **continue** skips to the next iteration.
- **Nested loops**: the outer loop controls rows, the inner loop controls columns.
- Always make sure the condition can become false, otherwise the loop is infinite.
- Useful tricks: **% 10** gets the last digit and **/ 10** removes it.

Loops become easy with practice. Dry run every program on paper first, then type it and run it.